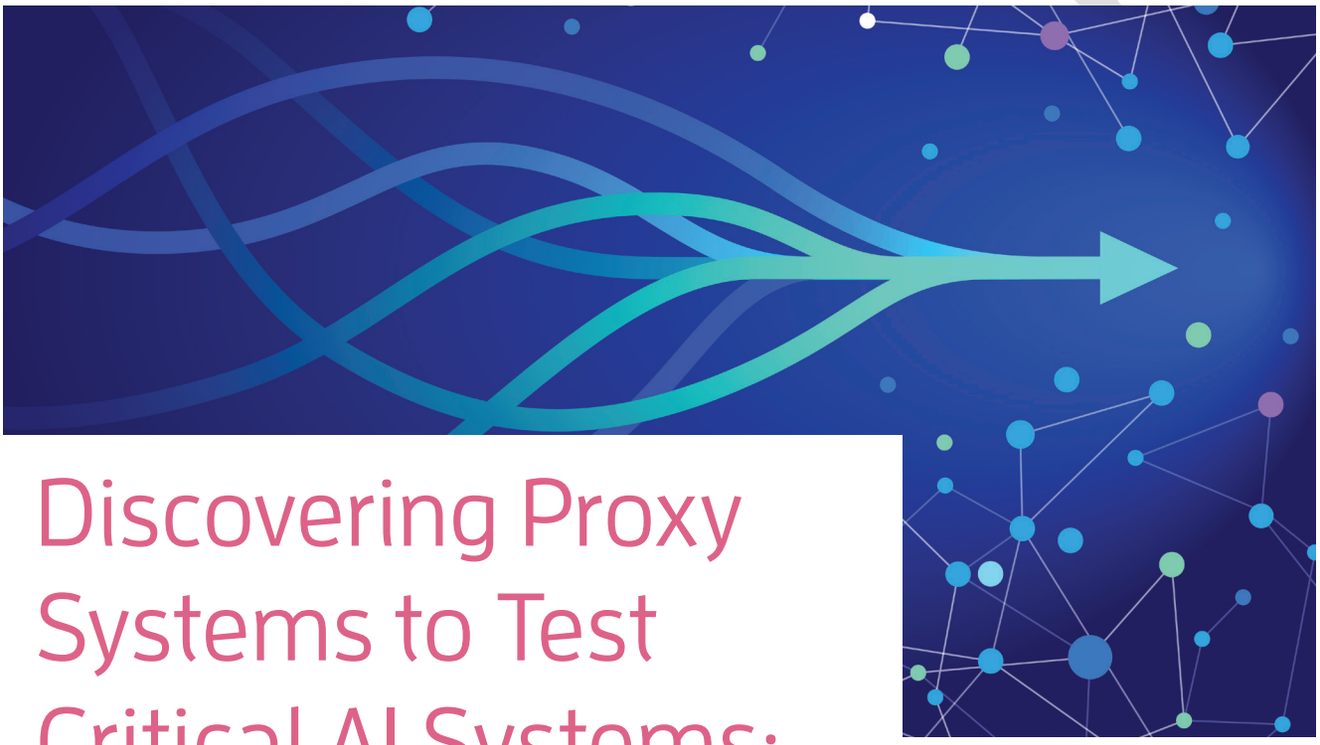

COPYRIGHT ISTOCKPHOTO, CREDIT:LVCANDY

Discovering Proxy Systems to Test Critical AI Systems: A Metadata-Driven Software Similarity Approach

Mark Kennedy, Joanna F. DeFranco, and Phillip A. Laplante, The Pennsylvania State University **<AU: Should Phillip A. Laplante's affiliation be listed as "IEEE Fellow"?>**

This article describes a practical approach for finding "software twins," noncritical proxy systems that mirror critical artificial intelligence-enabled systems and can be safely abused during testing, even when proprietary source code cannot be accessed for direct comparison.

Trustworthy artificial intelligence (AI) is the goal across industries. How do we thoroughly test AI-enabled software? This is becoming more difficult, especially in critical systems, where edge cases are too expensive or too dangerous to test. As we integrate AI into critical systems such as health care, transportation, finance, and public services, it becomes increasingly challenging to test the edge cases of these systems safely. It is critical to understand how these systems behave under stress, in rare edge cases, and when things go wrong. But we often cannot run those scenarios on the live system without putting people, infrastructure, or reputations at risk.

Digital Object Identifier 10.1109/MC.2026.3653167

0018-9162 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies.

EDITORS
JEFFREY VOAS IEEE Fellow;
j.voas@ieee.org
BENOIT BAUDRY Université de Montréal;
benoit.baudry@umontreal.ca

IEEE Proof

One promising approach is to leverage *proxy systems*: noncritical software that behaves similarly to the real system, which we can stress, break, or abuse during testing scenarios.[1,5] The challenge is in finding an acceptable proxy match for any critical AI system (CAIS), especially when we do not have access to proprietary source code.

In this article, we describe how a mix of AI, software engineering metrics, and simple Git logs can help to discover "software twins" that are good candidates to stand in for CAISs during testing. Since we do not have code to evaluate, we look for proxy matches through human and technical fingerprints left behind in public repositories: commit messages, collaboration patterns, file histories, and development rhythms.[4]

WHY SOFTWARE SIMILARITY MATTERS

When developers discuss code similarity, they often refer to code clones: matching functions, copied modules, or near-duplicate snippets.[2,3] In high-stakes deployments where systems use AI techniques, for example, machine learning code (ML) to generate clinical risk scores that inform medical treatment or an autonomous vehicle control stack (for example, the software and supporting hardware interfaces that translate environmental understanding into safe steering, braking, and acceleration commands), thorough testing is especially challenging. If we were to use a testing approach that leverages similar or proxy systems, we would want to define what it means for two systems to be similar. We could do this by asking certain key questions. For example, Does the proxy system run in a similar environment and serve a similar purpose? Does the proxy fail in similar ways or carry similar risks to people and infrastructure? Was the proxy built using comparable AI algorithms and development practices?

If we can answer those questions, we can look for noncritical systems that *behave* like our CAIS and then use those as proxies. We can push them to failure, explore misuse and adversarial conditions, and study recovery without ever touching the production system.[1,5]

The National Institute of Standards and Technology (NIST) has proposed a five-dimensional taxonomy to characterize such systems: physical environment, application purpose, operational characteristics, AI/ML algorithms, and development techniques.[1,5] That structure gives us a way to say, "This CAIS behaves like *that* proxy" in a principled way, rather than relying on gut feel or vague domain similarity.

But to use the taxonomy effectively, we must know something about how candidate systems are built. And that is where current similarity identification techniques run into trouble.

HOW WE COMPARE SOFTWARE TODAY

We reviewed recent literature to determine the most prevalent techniques to determine software similarity. Three categories emerged: code-centric similarity, dynamic/behavioral methods, and deep and cross-language approaches (see Figure 1).

Code-centric similarity

Clone detection is one of the oldest and best-developed approaches, making up 70% of the literature. These approaches essentially compare code at the source level (for example, if these two functions do the same thing, etc.). Tools search for repeated or slightly modified code fragments within and across projects, from contiguous blocks (for example, functions) to higher-level structural clones across classes and modules.[2,3]

Other work focuses on design patterns and architectural motifs: reverse-engineering patterns from code, measuring their prevalence, and exploring how they correlate with maintainability and quality.[8]

These techniques are powerful when code is available. They help detect plagiarism, shared libraries, or refactored components and are widely used in both academia and industry.[2,3]

Dynamic and behavioral methods

Another set of approaches, making up 20% of the literature, focuses on runtime behavior: instrumenting systems

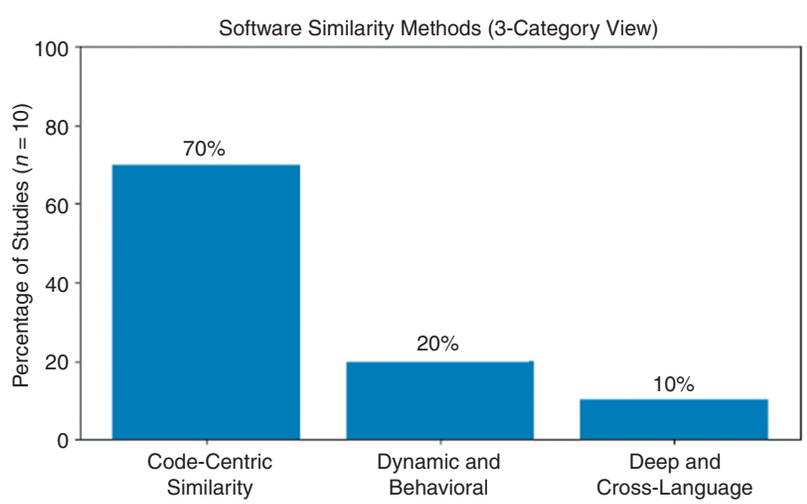

FIGURE 1. The distribution of software similarity methods in the reviewed software similarity subset.

and comparing execution traces or state transitions, building control flow and data flow graphs, and applying machine learning models to logs and runtime metrics to learn behavioral "fingerprints."[8] These are especially useful when binaries can be executed in controlled environments.

Deep and cross-language approaches

More recent work (probably why it only made up 10% of the literature), such as CroLSSim, combines structural and semantic representations, abstract syntax trees, latent semantic analysis-based features, and deep neural architectures, to compare code across different languages.[9] These hybrid approaches are particularly valuable when organizations need to reason about similarity across language or platform boundaries.

Across all these techniques, code access is necessary. For many everyday software engineering problems, that is perfectly reasonable. For CAISs, which are often classified or proprietary, code access is often unrealistic.

THE BLIND SPOT: WHEN CODE IS OFF LIMITS

For many CAIS scenarios, direct code access is not possible. The AI model may be proprietary or hosted by a third-party service, the system may mix commercial and open source components into closed box appliances, or regulatory and contractual constraints may limit reverse engineering or intrusive instrumentation.

At the same time, real-world incidents remind us that AI failures are not theoretical. The AI Incident Database, https://incidentdatabase.ai, for instance, catalogs numerous harms and near-harms caused by AI systems, including clinical decision support failures, biased risk scores, and safety-critical misclassifications.[4,6] Many of the systems in these reports are precisely the ones we would most like to probe, yet they are often the hardest to access or modify.

If proxy testing is going to be practical for CAISs, we need ways to identify and compare systems when we cannot see their code, cannot fully control their runtime environment, or only interact with them via application programming interfaces or logs. The good news is that even when code is off limits, we often still have rich metadata about how software evolves, especially in open source projects but increasingly in internal developer platforms as well.[4] Let us discuss some possibilities using Git metadata.

Reading between the commits: A metadata-first view

Modern development practices leave an extensive trail: Git histories, issue trackers, continuous integration logs, and deployment timelines. These artifacts paint a picture of how software evolves and how teams work two aspects that map directly to the "AI/ML algorithms" and "AI/ML development techniques," which are dimensions of the CAIS taxonomy.[5]

Our approach focuses on four classes of Git-based indicators: commit messages, contributor behavior, file change histories, and temporal evolution.[4] Together, these signals act like behavioral fingerprints for a software system.

Commit messages: Intent in plain text. Commit messages are where developers explain *why* a change was made, for example, "Add sepsis risk stratification model" or "Refactor authorization workflow for Medicaid appeals." Individually, any one message is noisy. Collectively, they tell a story about the domain, features, and evolution of the system.

Natural language processing lets us embed these messages in a semantic space and measure similarity between projects. If the commit message corpus of a candidate proxy has high semantic overlap with the CAIS's corpus, there is a good chance they serve related purposes and handle similar concepts.[4,9] For example, two projects that both contain dense clusters of messages about "vital signs," "risk scores," and "alert fatigue" are likely closer twins than one that mostly talks about "payment rails" and "fraud detection."

Contributor behavior: Sociotechnical signatures. A second layer of information comes from *who* is committing, to *what*, and *when*. Developer-to-file graphs, authorship entropy, and "bus factor" metrics distinguish projects built by a tightly knit core team from those maintained by broad communities or vendor-heavy teams. These patterns correlate with development techniques and sociotechnical risk topics that reliability researchers have highlighted in the context of rapidly converging technologies.[4,7] When we align projects based on these sociotechnical signatures, we are effectively matching the development techniques dimension of the NIST CAIS taxonomy.

File change histories: Architecture by cochange. Even without reading file contents, we can learn a lot from *which* files change together. Cochange graphs and churn metrics reveal a hidden architectural structure: modules often edited in tandem are likely coupled, files with high churn may host volatile logic, and stable core components often sit at the center of these graphs.[3,4]

Comparing such patterns between projects gives us a proxy for underlying architectural and algorithmic choices, similar to how structural clones expose deeper relationships in traditional clone detection work.[2,3,8]

Temporal evolution: The rhythm of development. Finally, we treat the project's history as a time series. Commit frequency, burstiness, release patterns, and long quiet intervals all encode how a team works. Techniques like time series clustering and dynamic time warping allow us to compare these rhythms between projects.[4] Agile teams may show regular cadence and frequent merges; research prototypes may show sporadic flurries of work. Mapping these patterns

IEEE Proof

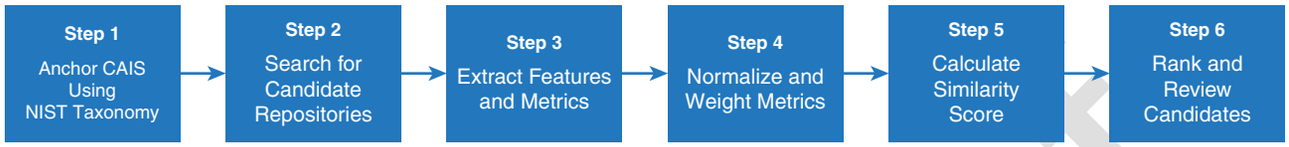

FIGURE 2. The similarity scoring process.

again helps us align projects along the development techniques dimension rather than just the operational environment.

SCORING SIMILARITY AT REPOSITORY SCALE

Each of these four indicator families gives us a partial view. The real power comes from combining them into a composite similarity score.[4] A repository similarity scoring system is proposed (Figure 2).

In step 1 of the process, we anchor the CAIS using the five-dimensional NIST taxonomy: we describe where it operates, what it does, what risks it poses, which AI/ML algorithms it uses, and how it is developed.[1,5]

In step 2, candidate repositories are collected by searching widely for systems in related domains, open source healthcare decision support tools, robotic control stacks, or financial risk engines, where at least Git metadata are available.

In step 3, next feature vectors are computed for each candidate with a semantic embedding of commit messages, graph-based features that summarize contributor behavior, co-change and churn metrics for the file graph, and time series descriptors of commit activity.

Step 4 normalizes and weights the metrics. Min–max normalization places each metric on a common scale. We assign weights reflecting which dimensions matter most for a given proxy task, for instance, heavier weight on commit semantics when functional purpose is critical or on contributor behavior when development technique is the focus.

In step 5, those weighted metrics are aggregated into a similarity score for each CAIS–candidate pair. The result is a tunable similarity value that can emphasize function, development practice, or architecture as needed.[4]

Finally, in step 6, we rank and review the candidates. The top-ranked systems receive deeper expert scrutiny and, when possible, more detailed architectural or code inspection.

Consider two candidate proxy systems that both resemble an autonomous vehicle CAIS but in different ways: one closely matches the commit content and development rhythm but is built by a very different kind of development team (for example, more/less organized, using a different methodology, etc.); the other is maintained by a team whose behavior strongly matches the CAIS's organization but whose commit content is less aligned. With different weights, either candidate might come out on top. That is a feature: to explicitly highlight the aspects of "similarity" that matter most for a particular testing campaign.

SIMILARITY-TO-PROXY SAFETY LOOP

Similarity scores alone do not guarantee a good proxy. They do, however, dramatically narrow the search space. As shown in Figure 3, candidates are mapped back to the CAIS taxonomy.

If a candidate shares the same operational environment, purpose, and development techniques but relies on a slightly different algorithm family, it may still be selected as a safe proxy for certain failure modes.[1,5]

Next, we can design targeted test campaigns around selected proxies. Using a proxy to test a CAIS, we can explore edge case behaviors that would be

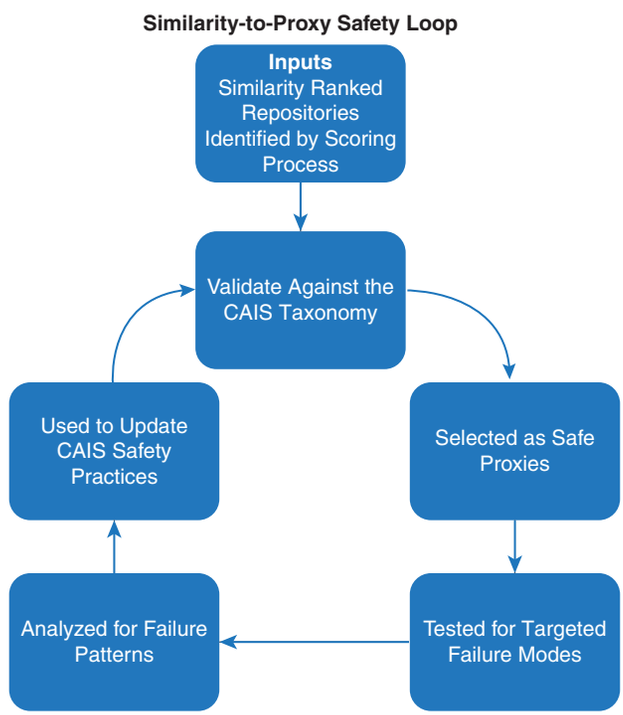

FIGURE 3. Feeds back into the subsequent proxy selection cycle.

IEEE Proof

unacceptable on the live CAIS: biased inputs, adversarial examples, infrastructure failures, or degraded sensors.

We can then transfer insights back to the CAIS. Failure patterns discovered in the proxy can inform risk assessments, monitoring strategies, and mitigation plans for the real system, even when we never run those exact tests in production.

Consider a clinical AI deployed in hospital triage. Real-world incident reports have shown that such systems can misallocate resources, misestimate risk for certain populations, or unintentionally encode discriminatory policies.[4,6] By finding a noncritical proxy, perhaps an open source risk-scoring tool used in a different domain but developed with similar methods, we can probe those types of failures in depth without touching the production system.[4]

Because this approach does not require direct access to the CAIS code, it scales scenarios where multiple vendors, regulators, and operators all need evidence of safety but cannot freely exchange proprietary implementations.[4,5]

<AU: Please identify the concluding paragraph. Per style It should begin with a drop cap, not a heading.>

LOOKING AHEAD

Proxy systems will not replace traditional verification and validation, but they can make high-risk AI testing much more practical. This fits squarely within the broader reliability engineering agenda of designing systems to be dependable, secure, and resilient in the face of rapidly converging technologies.[5,7]

There are three possible ways to validate the similarity process and scoring method calibration against known pairs, governance frameworks, and tooling for practitioners.

Calibration against known pairs

We need benchmark collections of systems where we *do* know the ground truth similarity, both functionally and architecturally. These can be used to tune weights, thresholds, and feature choices for our composite similarity scores.[4]

Integration with governance and assurance frameworks

Proxy selection should not be an informal, ad hoc exercise. As critical AI standards evolve, we can embed metadata-driven similarity into broader processes for risk management, certification, and continuous monitoring.[5]

Tooling for practitioners

Ultimately, we would like safety engineers and auditors to be able to point a tool at a CAIS repository (or its limited metadata) and receive a curated list of candidate proxies, along with explainable similarity metrics.

The core idea is simple: if we cannot see the code, we should pay close attention to the traces it leaves behind. Git histories, developer networks, and architectural fingerprints carry enough information to discover software twins at scale. When those twins can serve as safe proxies for testing, we gain a powerful new lever for understanding and managing the risks of CAISs.

REFERENCES

1. J. F. DeFranco, M. Kassab, and P. A. Laplante, "A taxonomy of critical AI system characteristics for use in proxy system testing," in *Proc. IEEE Int. Symp. Softw. Rel. Eng. Workshops (ISSREW)*, Piscataway, NJ, USA: IEEE Press, 2022, pp. 342–346, doi: 10.1109/ISSREW55968.2022.00090.
2. S. Jarzabek, "Software similarity patterns and clones: A curse or blessing?" in *Proc. 22nd Int. Conf. Enterprise Inf. Syst. (ICEIS)*, 2020, pp. 5–17, doi: 10.5220/0009820000050017.
3. S. Jarzabek, "From simple to structural clones: Tapping the benefits of non-redundancy," in *Proc. Int. Conf. Enterprise Inf. Syst.*, J. Filipe, M. Śmiałek, A. Brodsky, and S. Hammoudi, Eds. Cham, Switzerland: Springer-Verlag, 2021, pp. 563–590, doi: 10.1007/978-3-030-75418-1_26.
4. M. Kennedy, J. F. DeFranco, and P. A. Laplante, "Pattern discovery in critical AI failures: Toward proxy systems for safe testing," submitted for publication.
5. P. A. Laplante, J. F. DeFranco, D. R. Kuhn, and J. Voas, 2024, "Proxy validation and verification for critical AI systems: A proxy design process," Nat. Inst. of Standards and Technol., Gaithersburg, MD, USA. [Online]. Available: https://nvlpubs.nist.gov/nistpubs/CSWP/NIST.CSWP.31.pdf
6. "AI incident database (AIID)." Partnership on AI. Accessed: Dec. 25, 2025. [Online]. Available: https://incidentdatabase.ai
7. S. W. Shieh, "Reliability engineering in a time of rapidly converging technologies," *IEEE Trans. Rel.*, vol. 73, no. 1, pp. 73–82, Mar. 2024, doi: 10.1109/TR.2024.3355905.
8. D. Shilintsev and G. Dlamini, "A study: Design patterns detection approaches and impact on software quality," in *Proc. Int. Conf. Frontiers*

MARK KENNEDY is a doctoral student in the D.Eng. program at The Pennsylvania State University, University Park, PA 16802 USA. Contact him at mdk5293@psu.edu.

JOANNA F. DEFRANCO is an associate professor of software engineering and associate director of the D.Eng. program at The Pennsylvania State University, University Park, PA 16802 USA, and an associate editor in chief of *Computer*. Contact her at jfd104@psu.edu.

PHILLIP A. LAPLANTE is a professor emeritus of software and systems engineering at The Pennsylvania State University, University Park, PA 16802 USA, and editor in chief of *IEEE Reliability Magazine*. Contact him at plaplante@psu.edu. **<AU: Please confirm that the bio information is correct/current.>**

IEEE Proof

Softw. Eng., G. Succi, P. Ciancarini, and A. Kruglov, Eds., Cham, Switzerland: Springer-Verlag, 2022, pp. 84–96, doi: 10.1007/978-3-030-93135-3_6.

9. F. Ullah, M. R. Naeem, H. Naeem, X. Cheng, and M. Alazab, "CroLSSim: Cross-language software similarity detector using hybrid approach of LSA-based AST-MDrep features and CNN–LSTM model," *Int. J. Intell. Syst.*, vol. 37, no. 9, pp. 5768–5795, 2022, doi: 10.1002/int.22813.

IEEE Proof

If we were to use a testing approach that leverages similar or proxy systems, we would want to define what it means for two systems to be similar.

These hybrid approaches are particularly valuable when organizations need to reason about similarity across language or platform boundaries.

Many of the systems in these reports are precisely the ones we would most like to probe, yet they are often the hardest to access or modify.

Natural language processing lets us embed these messages in a semantic space and measure similarity between projects.

Mapping these patterns again helps us align projects along the development techniques dimension rather than just the operational environment.

Git histories, developer networks, and architectural fingerprints carry enough information to discover software twins at scale.

IEEE Proof